IFCD107 · ESPECIALISTA EN INTELIGENCIA ARTIFICIAL

Transformers y atención: cómo lee un LLM

Teoría, práctica y, al final, material de consulta

Sprint 12 · S12_05 · teoría breve (15 min el 24/11), 40 min de práctica en clase y material de consulta

LA IDEA · QUÉ HACE UN LLM

Un LLM solo sabe elegir el siguiente token

1 · Trocear

```
El|desayuno|estaba
```

El texto se parte en **tokens** y cada token es un número.

→

2 · Atender

```
cada token mira a
los demás
```

La **atención** mezcla en cada token lo que dicen los otros.

→

3 · Puntuar

```
un número por
candidato
```

Cada token posible recibe una puntuación: el **logit**.

→

4 · Elegir

```
softmax y
temperatura
```

Se pasan a probabilidades y se elige un token.

Elegido el token, se añade al texto y todo se repite: así escribe un LLM, token a token.

Un Transformer

Apila **muchas capas** de atención, con varias «cabezas» a la vez. Hoy calculamos una, a mano.

No es magia

Son **matrices y probabilidades**: por eso se puede calcular con **numpy**.

LA IDEA · TOKENS

Tokens: ni palabras enteras ni letras sueltas

Por palabras

Un vocabulario **enorme**, y ninguna respuesta ante una palabra nueva.

Por letras

Vocabulario diminuto, pero «habitaciones» son **12 símbolos**: secuencias larguísimas.

BPE: el punto medio

Parte de letras y **fusiona los pares más frecuentes** hasta formar trozos útiles.

Qué es un token

Un trozo frecuente de texto: a veces una palabra, a veces una raíz o una terminación.

Por qué importa

Los LLM cobran y se limitan **por tokens**. El español suele gastar más tokens que el inglés.

Un token no es una palabra: es un trozo de texto, y cada trozo es un número.

LA IDEA · BPE DE JUGUETE

BPE: cada fusión junta el par más frecuente

Paso	Par más frecuente	Veces	Símbolo nuevo
1	p + l	3	pl
2	pl + a	3	pla
3	pla + y	3	play
4	j + u	3	ju
5	ju + g	3	jug

Palabra	Tras 5 fusiones	Letras → tokens
playa	play · a	5 → 2
playas	play · a · s	6 → 3
playero	play · e · r · o	7 → 4
jugar	jug · a · r	5 → 3
jugamos	jug · a · m · o · s	7 → 5
jugaba	jug · a · b · a	6 → 4
jugando (nueva)	jug · a · n · d · o	7 → 5

36 letras → 21 tokens, sin saber español: han aparecido las raíces **play** y **jug**. Una palabra nueva se trocea con los mismos trozos.

LA IDEA · AUTOATENCIÓN

Atención: cada palabra decide a quién mirar

Q · query («pregunta»)

Lo que esta palabra **busca**.

K · key («etiqueta»)

Lo que cada palabra **ofrece**.

V · value («contenido»)

La información que **se pasa**.

«El desayuno estaba frío»

Para entender «frío» conviene mirar a «desayuno»: la atención deja que cada palabra decida **cuánto mira** a las demás.

Palabra	Q	K	V
desayuno	(2, 1)	(1, 2)	(3, 0)
estaba	(0, 1)	(2, 0)	(1, 1)
frío	(1, 3)	(0, 1)	(0, 2)

La Q de «frío», (1, 3), y la K de «desayuno», (1, 2), son las que mejor encajan.

LA IDEA · LOS CUATRO PASOS

Cuatro pasos para una palabra: «frío»

Paso	desayuno	estaba	frío
1 · Puntuaciones $Q \cdot K^T$	7	2	3
2 · Escalar ÷ √2	4,95	1,41	2,12
3 · Softmax (pesos)	**0,919**	0,027	0,054

4 · Mezclar

salida de «frío» = **0,919**·V(desayuno) + 0,027·V(estaba) + 0,054·V(frío)

= **(2,784 ; 0,135)**

¿Por qué ÷ √d?

Aquí d = 2, y √2 ≈ 1,41. Se divide para que los números **no crezcan demasiado** antes del softmax.

«frío» mira sobre todo a «desayuno» y su vector ya lleva información de él.

Softmax: de puntuaciones a probabilidades

Opción	Puntuación **x**	**exp(x)**	Probabilidad
a	4	54,6	0,867
b	2	7,4	0,117
c	0	1,0	0,016
Suma		63,0	**1,000**

Truco: restar el máximo

x – máximo: (4, 2, 0) pasa a (0, −2, −4) y las probabilidades salen **iguales**.

Por qué hace falta

Con (1 000, 998, 996), **exp(1000)** desborda: sale **inf** y el resultado es **nan**. Restando el máximo, salen las mismas 0,867 · 0,117 · 0,016.

Por filas: cada fila de la matriz de atención suma 1. El error típico es poner el eje al revés.

Máscara causal: no mirar al futuro

Sin máscara

mira a →	desayuno	estaba	frío
desayuno	0,47	0,47	0,06
estaba	0,58	0,14	0,28
frío	0,92	0,03	0,05

Con máscara causal

mira a →	desayuno	estaba	frío
desayuno	1,00	0,00	0,00
estaba	0,80	0,20	0,00
frío	0,92	0,03	0,05

Qué se hace

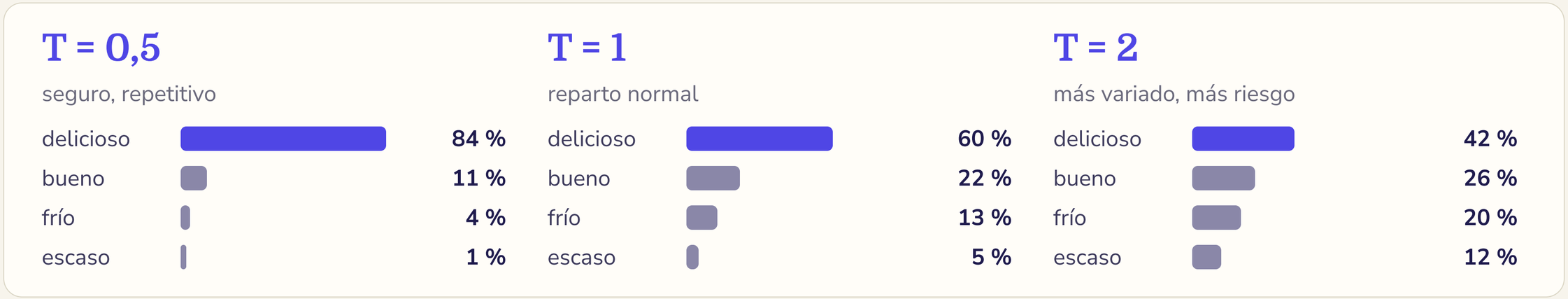
Antes del softmax, un número muy negativo (**−1e9**) por encima de la diagonal: el futuro. Su peso pasa a **0**.

Qué se ve

La primera palabra solo se mira a sí misma (**1,00**). La **última fila no cambia**: ya veía todo lo anterior.

LA IDEA · TEMPERATURA

Temperatura: cuánto arriesga el modelo al elegir



T baja

Casi siempre el token **más probable**: respuestas previsibles.

T alta

Más reparto: **más variedad** y más riesgo de errores o alucinaciones.

Siguiente token de «El desayuno estaba…». Mismo modelo, mismos logits: solo cambia el reparto.

PRÁCTICA · CUADERNO S12_05

Lo que haremos en el cuaderno

Parte	Qué haremos	Dónde
Preparación	Preparar las herramientas y contar pares	Clase
Ejercicios 1 y 2	BPE: par más común y compresión	Casa
Ejemplo	Q, K y V con tres palabras	Clase
Ejercicio 3	Escribir softmax por filas	Clase
Ejemplo	Pasos 2 y 3: escalar y softmax	Clase
Ejercicio 4	La función de atención completa	Clase
Ejercicio 5	Atención con máscara causal	Clase
Ejercicio 6	Temperatura: probabilidad con T baja y alta	Casa
Mini-reto	Que «hotel» preste más atención a «limpio»	Casa
Opcional	Modelos de Hugging Face (con internet)	Casa

Cada ejercicio: **lee** → **escribe** → **ejecuta** → **comprueba el** ✔. La solución está plegada: ábrela tras intentarlo.

CONSULTA

Material de consulta: código y aclaraciones

Cuándo abrirlo

Después de intentar el ejercicio, o cuando algo no te salga. En clase no se proyecta.

Cómo está ordenado

En el **mismo orden que el cuaderno**, cada ejercicio con su código y sus aclaraciones.

Ejercicio	Tema	Diapositivas
1	El par de letras más común	12 y 13
2	Razón de compresión del BPE	14 y 15
3	Softmax por filas	16 y 17
4	La atención completa	18, 19 y 20
5	Atención con máscara causal	21 y 22
6	Probabilidad máxima y temperatura	23 y 24
Mini-reto	Que «hotel» mire más a «limpio»	25 y 26

CONSULTA · EJERCICIO 1

El par de letras más común

```
pares = contar_pares([list(p) for p in corpus])
par_mas_comun = pares.most_common(1)[0][0]
```

('d', 'e')

par_mas_comun

4

Veces que aparece

Aparece en «desayuno», «desayunos», «desayunar» y «desayunamos»: el primer par que fusiona el BPE.

CONSULTA · ACLARACIONES · EJERCICIO 1

Leer el par más común (ejercicio 1)

contar_pares([list(p) for p in corpus])

Cuenta cada par de símbolos **consecutivos** en todas las palabras.

most_common(1)[0][0]

El primer [0] toma el par con su cuenta; el segundo deja **solo el par**.

Resultado

par_mas_comun = **('d', 'e')**, **4** veces.

Dónde aparece

En «desayuno», «desayunos», «desayunar» y «desayunamos».

CONSULTA · EJERCICIO 2

Razón de compresión del BPE

```
n_tokens_bpe = sum(len(t) for t in tokens)
n_letras = sum(len(p) for p in corpus)
compresion = round(n_letras / n_tokens_bpe, 2)
```

67 → 22

Letras → tokens

3,05×

Compresión

Con 12 fusiones el BPE descubre raíces como **desayun** y **limpi sin saber español**.

CONSULTA · ACLARACIONES · EJERCICIO 2

Leer la compresión (ejercicio 2)

sum(len(t) for t in tokens)

Cuenta los **tokens** tras las fusiones, no las palabras.

sum(len(p) for p in corpus)

Cuenta las **letras** del corpus original.

Resultado

67 letras → **22** tokens: compresión **3,05**.

Por qué importa

Los LLM cobran y se limitan **por tokens**; el español gasta más que el inglés.

CONSULTA · EJERCICIO 3

Escribir softmax por filas

```
def softmax(m):
    e = np.exp(m - m.max(axis=1, keepdims=True))
    return e / e.sum(axis=1, keepdims=True)
```

1/3 · 1/3 · 1/3

softmax([[0., 0., 0.]])

suma 1

Cada fila del resultado

Se resta el máximo de cada fila: evita desbordes y el resultado **no cambia**.

CONSULTA · ACLARACIONES · EJERCICIO 3

Leer el softmax (ejercicio 3)

`m - m.max(axis=1, keepdims=True)`

Resta el **máximo de cada fila**: evita desbordes y el resultado **no cambia**.

`e.sum(axis=1, keepdims=True)`

Suma **por filas**: cada fila del resultado suma **1**.

Resultado

`softmax([[0., 0., 0.]])` da **1/3 · 1/3 · 1/3**: entradas iguales, probabilidades iguales.

Error típico

Sumar por columnas o sobre toda la matriz: el eje va **al revés**.

CONSULTA · EJERCICIO 4

La función de atención completa

```
def atencion(X, Wq, Wk, Wv):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    pesos = softmax(Q @ K.T / np.sqrt(K.shape[1]))
    return pesos @ V, pesos
```

(3, 2)

Forma de la salida

(0,66 ; 3,78)

Fila de «hotel»

Cada fila de la salida es una **mezcla** de los contenidos V de las tres palabras.

CONSULTA · ACLARACIONES · EJERCICIO 4

Leer la atención (ejercicio 4)

Q, K, V = X @ Wq, X @ Wk, X @ Wv

Cada palabra se proyecta a su **pregunta**, su **etiqueta** y su **contenido**.

softmax(Q @ K.T / np.sqrt(K.shape[1]))

Puntuaciones, **escaladas** por √d y pasadas a probabilidades por filas.

Resultado

Salida de forma **(3, 2)**; la fila de «hotel» es **(0,66 ; 3,78)**.

Cuidado

Devolver **pesos** en lugar de **pesos @ V**: la salida es una **mezcla de contenidos**.

La matriz de pesos: quién mira a quién

Pesos de atención del cuaderno

mira a →	hotel	muy	limpio
hotel	0,11	0,45	0,45
muy	0,05	0,89	0,05
limpio	0,09	0,73	0,18

Cómo se lee

Fila: la palabra que pregunta. **Columna**: a quién mira. Cada fila suma **1**.

Palabra	Salida = pesos · V
hotel	(0,66 ; 3,78)
muy	(0,16 ; 3,89)
limpio	(0,35 ; 3,82)

atencion(X, Wq, Wk, Wv) devuelve **(salida, pesos)**: una mezcla de contenidos por palabra.

CONSULTA · EJERCICIO 5

Atención con máscara causal

```
def atencion_causal(X, Wq, Wk, Wv):
    Q, K, V = X @ Wq, X @ Wk, X @ Wv
    p = Q @ K.T / np.sqrt(K.shape[1])
    mascara = np.triu(np.ones_like(p), k=1) == 1
    p[mascara] = -1e9
    pesos = softmax(p)
    return pesos @ V, pesos
```

1 · 0 · 0

Primera fila

0,06 · 0,94 · 0

Segunda fila

Enmascarar **antes** del softmax; después, las filas ya no sumarían 1.

CONSULTA · ACLARACIONES · EJERCICIO 5

Leer la máscara causal (ejercicio 5)

np.triu(np.ones_like(p), k=1) == 1

Marca lo que está **por encima de la diagonal**: las palabras futuras.

p[mascara] = -1e9

Antes del softmax: **exp(-1e9) ≈ 0**, así el peso es **0** y las filas **siguen sumando 1**.

Resultado

Filas **1 · 0 · 0**, **0,06 · 0,94 · 0** y **0,09 · 0,73 · 0,18**.

Cómo leerlo

La última fila **no cambia**: ya veía todo lo anterior.

CONSULTA · EJERCICIO 6

Probabilidad máxima con T baja y alta

```
prob_max_T03 = float(softmax(logits / 0.3).max())
prob_max_T25 = float(softmax(logits / 2.5).max())
mas_seguro = "T baja" if prob_max_T03 > prob_max_T25 else "T alta"
```

0,816

T = 0,3 · prob. máxima

0,302

T = 2,5 · prob. máxima

mas_seguro = «T baja»: concentra la probabilidad en lo más probable.

CONSULTA · ACLARACIONES · EJERCICIO 6

Leer la temperatura (ejercicio 6)

softmax(logits / 0.3).max()

Se divide por T **antes** del softmax: con T baja la distribución es **puntiaguda**.

mas_seguro

"T baja" si prob_max_T03 > prob_max_T25.

Resultado

0,816 con T = 0,3 y **0,302** con T = 2,5.

Qué elegir

Para datos de reservas, T **baja**: respuestas predecibles y pocos errores.

CONSULTA · MINI-RETO

Que «hotel» preste más atención a «limpio»

```
X2 = X.copy()
X2[2] = X2[2] * 2
_, pesos_nuevos = atencion(X2, Wq, Wk, Wv)
subio = bool(pesos_nuevos[0, 2] > pesos[0, 2])
```

0,446 → 0,932

Atención de «hotel» a «limpio»

True

subio

Al duplicar el embedding de «limpio», su K crece y la puntuación de «hotel» hacia «limpio» sube.

CONSULTA · ACLARACIONES · MINI-RETO

Leer el mini-reto

X2[2] = X2[2] * 2

Duplica el embedding de «limpio»: su **K** crece y la puntuación **Q·K** de «hotel» sube.

pesos_nuevos[0, 2] > pesos[0, 2]

Compara la atención de «hotel» a «limpio» **antes y después**.

Resultado

De **0,446** a **0,932**; **subio** = **True**.

Qué más controla

Las matrices **Wq** y **Wk**, que un modelo real aprende al entrenar.